

FORGE FDE ACADEMY · DATA & METRICS STUDIO

The dbt Semantic Layer

Consistent Metrics at Scale

Two dashboards, one metric, two different numbers. That board meeting is a symptom — the cause is architectural. A semantic layer defines the metric once, and serves it everywhere.

Semantic Models

MetricFlow

Entities · Dimensions · Metrics

Time Spine

THE PROBLEM

Semantic drift — the same metric, computed three ways, all in production



The scene every data team recognises: Finance builds a dashboard. Sales builds another. Both show “revenue”. The numbers disagree — not because anyone made a mistake, but because each team re-derived the definition in their own BI tool, because there was nowhere to put it once. Two dashboards, one word, two truths. The meeting stalls arguing about which chart is right.

WITHOUT a semantic layer

raw data → transform → **each BI tool defines metrics itself**

- Every consumer re-implements “revenue” — and they drift apart.
- The logic lives in dashboards, notebooks, and one analyst's head.
- No single place to fix, review, or govern a definition.
- Onboard a new BI tool → re-derive every metric again.

WITH a semantic layer

raw data → transform → **semantic layer** → every tool

- The metric is defined once, in one governed place.
- Power BI, Tableau, Sheets, Excel and the AI agent all read the same definition.
- Change it once; every consumer updates. Review it like code.
- Being consistent is now easier than being inconsistent.

The one-line definition: a semantic layer is an abstraction between your data models and the tools that consume them, where metric logic — metrics, dimensions, and the relationships between them — is defined once and reused everywhere. You stop calculating the same metric again and again in every BI tool.

THE LANDSCAPE

How we got here — three generations of semantic layer



BI-tool semantic layers

Tableau, Power BI — a semantic layer defined mostly in the UI: click, drag, configure.

✓ Familiar, no extra service, close to the report.

✗ Lives inside one tool. Hard to version-control, review, or reuse across other tools.



LookML (Looker)

Looker's modelling language — the first big step to defining metrics as CODE.

✓ Version control, CI/CD, software-engineering practices applied to metrics. Mature and popular.

✗ Usable only within Looker. An API exists, but access from other tools is limited — you are in the Looker ecosystem.



External layers — dbt · Cube · AtScale

A semantic layer as an independent service, defined in code (YAML), decoupled from any one BI tool.

✓ Define once; connect many tools — Power BI, Tableau, Hex, Sheets, Excel. Not locked to a single BI vendor.

✗ A component to run and govern in its own right; needs real cross-team agreement to populate.

The through-line of all three generations: the direction of travel is toward defining metrics as versioned code, decoupled from any single consumer. LookML proved metrics-as-code; the external layers freed it from one BI tool. The trade you make going right is that you gain reuse and governance but take on a component to run — which is why the small-team advice later in this deck is “not yet”. The prize is one source of truth for every number the business quotes.

01



Anatomy of the dbt Semantic Layer

MetricFlow · semantic models · entities · dimensions · metrics · time spine. The parts, before we assemble them.

ANATOMY · 1 OF 3

dbt transforms; MetricFlow defines the metrics on top



dbt — the transformation layer beneath

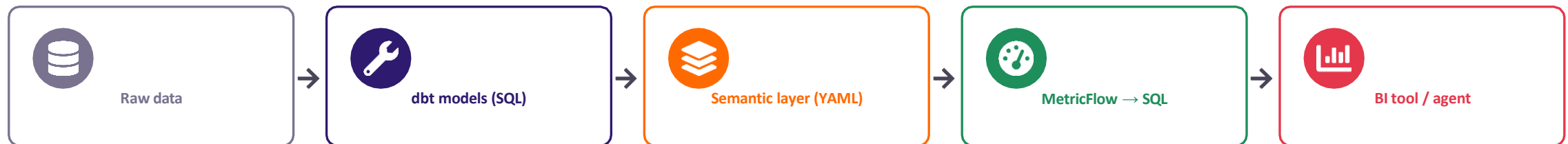
- Transforms raw data into clean, final data models (“marts”) using SQL.
- Transformations in SQL; configuration in YAML.
- Version-controlled, testable, CI/CD-friendly — the reason metrics-as-code works.
- The semantic layer sits on top of the gold/marts models, not the raw data.



MetricFlow — the engine that resolves them

- You declare metrics in YAML; MetricFlow reads the config and generates the SQL.
- Came from Transform, a company dbt Labs acquired — replacing the earlier, clunkier “dbt metrics”.
- Handles joins, aggregation and time-based slicing under the hood.
- Re-open-sourced under Apache 2.0 — it is the reference implementation others build on.

The metric request, end to end



Why this ordering matters: the semantic layer is downstream of your dbt models and upstream of every consumer. It is not a replacement for transformation — it is a governed contract for meaning that sits between the clean tables and the tools. Point an LLM at raw tables and it invents joins and re-derives metrics fluently and inconsistently. Point it at MetricFlow and it selects a governed metric; the engine writes correct SQL. **Accuracy becomes a property of the architecture, not the prompt.**

ANATOMY · 2 OF 3

Six parts — semantic model, entities, dimensions, metrics, saved queries, time spine



Semantic model

A 1:1 wrapper over one dbt data model. If you have an orders model, you create one semantic model for it, and declare everything else inside it.



Entities

Identifiers for joins — the primary key and the foreign keys. MetricFlow uses these under the hood to connect this model to others (e.g. orders → locations).



Dimensions

The fields you slice and dice by. Two kinds: TIME (e.g. ordered_at) and CATEGORICAL (e.g. is_food_order). This is how a metric is broken down.



Metrics

The KPI definitions themselves: a name, a type (simple, ratio, derived...) and an aggregation (sum, count, count-distinct) over a field. e.g. revenue = sum(order_total).



Saved queries

A frequently-used metric + dimension combination, saved as a reusable template so you do not re-specify it every time.



Time spine

A date table that powers time-based aggregation. Without it, a model with a time dimension will not work — it is what lets you roll up by day, week, month, quarter, year.

Mental model: the semantic model is the container; entities wire it to other models; dimensions are the axes; metrics are the numbers; saved queries are shortcuts; the time spine makes time work.

ANATOMY · 3 OF 3

dbt Core vs the dbt platform — where the semantic layer actually runs



dbt Core — open source, free

- You CAN define metrics — MetricFlow is open source.
- But: no built-in native BI integrations, and no hosted API for tools to connect to.
- To consume metrics you build the API yourself (e.g. in Python) — or reach for another open tool.
- Great for defining and validating; harder for serving to many consumers.



The dbt platform — paid

- Native integrations: Power BI, Tableau, Google Sheets, Excel, Hex, Omni, Sigma and more.
- A hosted Semantic Layer API — any tool can connect via drivers or the API.
- Exports and saved queries for tools without a native connector.
- The path most teams take if they want the SL served, not just defined.



The honest trade-off: the engine is free; the SERVING is what you pay for. dbt Core gives you MetricFlow and correct metric definitions, but turning those into something Power BI can query out-of-the-box is the platform's job — or a build-it-yourself API. Frame it to clients as define-vs-serve, and note it is exactly the kind of packaging boundary that is shifting with the merger and OSI — so verify before you quote it.

ACCURACY FLAG — read this to the room. Packaging here is volatile: the semantic-layer spec was rewritten and shipped in dbt Core v1.12; MetricFlow is now Apache-2.0; Fivetran and dbt Labs merged in June 2026. The define-vs-serve split is directionally true but the exact “what needs the paid platform” line moves — always confirm against current docs before advising a client.

02



Build-Along — Jaffle Shop

From a plain orders model to a governed metric, queried from Google Sheets. The same demo the workshop runs, distilled to the moves that matter.

BUILD · 1 OF 3

Define the orders semantic model — one YAML file, four parts

The project: the Jaffle Shop sample, with staging → intermediate → marts layers. We wrap the marts “orders” model in a semantic model and declare its entities, dimensions and metrics.

models/marts/orders.yml — the four declarations

```
semantic_models:
- name: orders_sl
  model: ref('orders')
  defaults: {agg_time_dimension: ordered_at}
  # 1 · ENTITIES — the join keys
  entities:
    - {name: order_id, type: primary}
    - {name: location_id, type: foreign}
    - {name: customer_id, type: foreign}
  # 2 · DIMENSIONS — slice & dice
  dimensions:
    - {name: ordered_at, type: time,
      type_params: {time_granularity: day}}
    - {name: is_food_order, type: categorical}
  # 3 · METRICS live alongside (see right)
```

...same file — the metrics

```
metrics:
- name: revenue
  type: simple
  type_params:
    measure: {agg: sum,
      expr: order_total}
- name: orders_count
  type: simple
  type_params:
    measure: {agg: count,
      expr: order_id}
```

What you just declared

- order_id is the primary key; location_id & customer_id are foreign keys → free joins later.
- revenue = sum(order_total); orders_count = count(order_id).
- Both are slice-able by time (ordered_at) and by is_food_order.

BUILD · 2 OF 3

Add the time spine, then validate with MetricFlow

The gotcha: a semantic model with a time dimension will not build without a time spine — a date table MetricFlow uses to roll metrics up by day, week, month, quarter and year.

models/marts/time_spine_daily.sql

```
-- one row per day; engine-specific SQL
{{ config(materialized='table') }}

with days as (
  -- generate_series / date spine
  -- 2000-01-01 .. 2030-01-01
)
select date_day from days
where date_day > '2020-01-01'
```

...and tell dbt it IS a time spine

```
models:
  - name: time_spine_daily
    time_spine:
      standard_granularity_column:
        date_day
    columns:
      - name: date_day
        granularity: day
```

Validate before you consume — MetricFlow commands

terminal

```
$ dbt parse           # compile config
$ dbt sl list metrics # what's defined?
$ dbt sl query \
  --metrics revenue --group-by metric_time
```

Read the output like a test

dbt sl list metrics prints each metric with the dimensions it can be sliced by. If revenue and orders_count appear with metric_time and is_food_order — the semantic model resolved correctly. If a metric is missing, the config did not parse. This is your unit test before any BI tool is connected.

BUILD · 3 OF 3

Join a second model, then consume from a BI tool



Revenue by location — no JOIN written

Want “revenue by location name”? You do not write a join. You give the **locations** model its own semantic model, declare **location_id** as its PRIMARY entity, and expose **location_name** as a dimension. Because orders already declared **location_id** as a FOREIGN entity, MetricFlow now resolves the join itself. Re-run `dbt sl list metrics` and revenue gains **location_name** as a new slice-by option.



Consume — commit, then connect

- Commit the YAML, open a PR, merge, and run the production job so the platform serves the latest definitions.
- In Google Sheets, add the Semantic Layer add-on, authenticate to the dbt platform, and the metrics appear in a query builder.
- Pick revenue, group by month (thanks to the time spine) and **location_name**, add a filter — query. Same definition a dashboard or an agent would get.

The payoff — one definition, many front doors

Google Sheets / Excel

Add-on → query builder → live metrics in a spreadsheet.

Power BI / Tableau

Native connectors read the same governed definitions.

Hex · Sigma · Omni

First-class partner integrations on the platform.

AI agent / app

Queries the metric via the API — no invented SQL, no drift.

The moment it clicks: you defined revenue exactly once, in reviewed YAML, and queried it from a spreadsheet — with month-level time aggregation and a location join you never hand-wrote. The next team that needs “revenue” does not rebuild it; they select it. That is the whole value proposition, demonstrated end to end in under an hour.

03



When It Wins — and When It Doesn't

Trade-offs, cost, and the honest guidance on when a semantic layer is premature. The organisational work is harder than the YAML.

TRADE-OFFS

What it costs — the hard part is human, not technical



Definition consensus

Every department must AGREE on one definition per KPI. This is negotiation, not engineering — and it is the real project. The YAML takes an afternoon; the agreement can take weeks.



Metric ownership

Each metric needs a named owner responsible for maintaining it and adjudicating change requests. An unowned metric decays into the next drift.



Warehouse compute

Queries generate SQL that runs on the warehouse — real compute, real cost. Governance is not free; every query has a bill attached.



Platform dependency

To SERVE the layer conveniently (API, native integrations) you lean on the paid platform — or you build and maintain the serving yourself.



THE HONEST HEADLINE

Implementing a semantic layer is not primarily a coding task — it is a governance task wearing a YAML costume. The tool will faithfully serve whatever definition you give it, which means the value is created in the room where Finance and Sales agree what “revenue” means — not in the editor. An FDE who ships the YAML but skips the consensus has built a very consistent way to serve a contested number. Budget for the meetings, name the owners, and treat the definitions as the deliverable.

THE DECISION

When to reach for it — and when “not yet” is the right answer



Reach for it when

- The same KPIs are used across multiple tools and multiple teams.
- You need one governed place to define and change metric logic.
- Numbers must agree across dashboards, reports, and increasingly an AI agent.
- You have the organisational will to agree definitions and name owners.



“Not yet” when

- Small team, early stage — your effort belongs in the foundational data models first.
- The metric is fixed and never sliced — just pre-compute it somewhere simpler.
- No cross-team disagreement exists yet — you would be governing a problem you do not have.
- Nobody will own the definitions — the layer will rot on day one.



The pattern most companies actually land on — hybrid. The company-wide “North Star” metrics — the ones that must be identical everywhere and survive an audit — live in the semantic layer with approved definitions and named owners. The long tail of local, department-specific metrics stays in the BI tools, where speed matters more than governance. You do not semantic-layer everything; you semantic-layer what must not disagree.

FDE rule of thumb: the trigger is not company size — it is the first time two people in the room quote different numbers for the same word. Before that, a semantic layer is governance you have not earned the need for. After it, every month you wait, the drift compounds.

04



The Landscape Is Moving

The syntax gotcha · Open Semantic Interchange · the AI-agent driver. Why the version you run and the standard you back both matter now.

GOTCHA

Match your syntax to your engine — the error that stops the demo

What happened in the workshop: the build failed at the production job — not because the metrics were wrong, but because the NEW semantic-layer syntax was run on an engine that did not yet support it. Switching to the Fusion engine fixed it instantly.

Legacy spec

Metrics and measures defined separately, across multiple files. What older tutorials show.

New spec (rewritten)

Semantic models embedded inside the model's own YAML; measures folded into simple metrics; common options promoted to top-level keys — a real simplification.

Where it runs

Live on the Fusion engine first; shipped to dbt Core in v1.12. Older Core versions need the legacy shape.

Migration path

dbt-autofix can migrate most legacy definitions to the new spec automatically — you rarely rewrite by hand.

The transferable lesson — and it is bigger than dbt: when a tool is evolving fast, the syntax in the docs, the syntax in a tutorial, and the syntax your installed engine accepts can be three different things. Before you demo to a client, confirm the engine version and the spec version match. “It worked in the video” is not a defence when the production job fails in the room. Pin versions; read the release notes; keep a fallback engine ready.

WHERE IT'S HEADING

Open Semantic Interchange — and why agents made all of this urgent



OSI — a portable format, not a new layer

Snowflake, Salesforce, dbt Labs, Cube, AtScale and many others are building a **vendor-neutral standard** for expressing semantic models and metrics — so a definition is portable between tools instead of locked in one. Think USB-C for semantics: not another device, a common connector. It does not replace dbt, Cube or LookML — it is a format they can all speak.

The FDE question to any vendor: “can you export your model to the OSI format today, and import someone else's?” That answer separates the press release from the product.



Why agents made this urgent

An AI agent pointed at raw tables invents joins and re-derives metrics — fluently and inconsistently. Pointed at a semantic layer, it selects a **governed** metric and the engine writes correct SQL. Warehouses are adding semantic constructs for exactly this — e.g. Snowflake's semantic views, which scope and define the data an agent may query, so the agent reasons over governed meaning rather than guessing.

The shift: the semantic layer stops being a BI convenience and becomes the governed context an AI reads. That is why every data vendor now wants to own it.



THE THROUGH-LINE OF THE WHOLE SESSION

A semantic layer began as an answer to a boardroom embarrassment — two dashboards, one metric, two numbers. It became a way to define metrics as versioned code and serve them to any BI tool. And now, because AI agents consume business data continuously and must not improvise definitions, it has become a piece of **AI architecture**. The consistent theme, from the drift problem to OSI to Snowflake's semantic views, is a single idea: **define meaning once, govern it in one place, and let everything else — humans and agents alike — be a consumer of that definition rather than an author of a new one.**

WRAP-UP

What you can do tomorrow that you could not do yesterday



Shipped this session

- Name the problem precisely: semantic drift is an architecture gap, not a discipline failure.
- Place the semantic layer: downstream of dbt models, upstream of every consumer, with MetricFlow as the engine.
- Build one end to end: semantic model, entities, dimensions, metrics, and the time spine that makes time work.
- Get joins for free by declaring entities — not by writing SQL.
- Validate with dbt parse and dbt sl list metrics before touching a BI tool.
- Judge when it wins, when to say “not yet”, and why hybrid is the usual landing.



Your assignment before next session

- Pick ONE metric your organisation computes two different ways. Write its single governed definition.
- Model it: one semantic model, its entities, one time and one categorical dimension, the metric, a time spine.
- Run dbt parse and dbt sl list metrics; confirm it resolves with its dimensions.
- Name the human owner of that definition — the deliverable is the agreement, not the YAML.
- Confirm your engine and spec versions match BEFORE you show anyone.

The through-line: the semantic layer is not really about metrics — it is about where meaning lives. Put a definition in a dashboard and it is true in that dashboard. Put it in a semantic layer and it is true everywhere, for every human and every agent that asks. Your job as an FDE is to find the numbers a business cannot afford to have disagree — and to move their meaning to one governed place, once, so nobody ever re-derives them again.

Define it once. Serve it everywhere.

Bring one metric your organisation computes two ways, its single governed definition, and the name of the person who will own it. We will model it, validate it, and connect it — and never re-derive it again.

Semantic Models

MetricFlow

Entities → free joins

Govern the meaning